

Tugas 1 - VM Manager

Sistem Operasi

Fakultas Ilmu Komputer, Universitas Indonesia

Gasal 2026/2027

Kelas : C

Kelompok : 10

Varian : A

Nama	NPM
Violin Monica	2506551794
Jihan Nabiilah Perma Sukma	2506549026
Rindu Maharani Nadhirah	2506587131
Stephanie Revalina Tamus	2506547153
Leow Vincent Vintizel	2506611856

Penjelasan

vm_ctl.sh

Penjelasan Program: **vm_ctl.sh** adalah script utama yang dijalankan dari komputer host untuk mengontrol VM di VirtualBox. Script ini memakai **VBoxManage**, yaitu command line tool dari VirtualBox, agar VM bisa dikendalikan lewat terminal bash seperti Git Bash atau WSL. Fitur yang ada di script ini mencakup melihat daftar VM, menampilkan informasi VM, menyalakan VM secara headless, mematikan VM dengan aman, membuat snapshot, dan melihat daftar snapshot. Setiap fitur dibuat dalam fungsi terpisah agar struktur program lebih rapi, mudah dibaca, dan sesuai dengan ketentuan tugas yang meminta script ditulis secara modular. Alur kerja program secara umum adalah sebagai berikut:

1. Menerima command dari user -> script membaca command yang diberikan saat dijalankan, misalnya **list**, **info**, **start**, **stop**, atau **snapshot**. Command tersebut kemudian diproses di dalam fungsi **main()** menggunakan percabangan **case**.
2. Memanggil fungsi sesuai command -> jika user menjalankan **./vm_ctl.sh list**, script akan memanggil fungsi **vm_list()**. Jika user menjalankan **./vm_ctl.sh info <nama_vm>**, script akan memanggil **vm_info()**. Pola yang sama juga digunakan untuk command **start**, **stop**, dan **snapshot**.
3. Menjalankan perintah **VBoxManage** -> setiap fungsi di dalam script menjalankan command **VBoxManage** sesuai kebutuhan. Contohnya, **VBoxManage list vms** digunakan untuk melihat daftar VM, **VBoxManage showvminfo** digunakan untuk melihat informasi VM, dan **VBoxManage snapshot** digunakan untuk mengelola snapshot.
4. Menampilkan hasil ke terminal -> setelah command dijalankan, script menampilkan output yang mudah dibaca oleh user, seperti daftar VM, informasi RAM dan vCPU, status VM, hasil start/stop VM, atau daftar snapshot.
5. Melakukan validasi sederhana -> beberapa command membutuhkan input tambahan seperti nama VM atau nama snapshot. Jika input belum lengkap, script akan menampilkan pesan error dan format command yang benar agar user tahu cara menjalankannya.

resource_check.c

Penjelasan Program: Program ini menerima 2 nilai metrik sistem (disk usage & memory usage) yang dikirim dari **sysinfo.sh** lewat pipe ke stdin lalu menentukan status PASS/WARN/FAIL tiap metrik berdasarkan ambang Varian A kemudian mengembalikan hasilnya balik ke [sysinfo.sh](#). Program ini dijalankan di dalam VM (guest) dan dikompilasi menggunakan GCC. Alur kerja program secara lengkap:

1. Membaca input dari stdin -> program menerima 2 nilai metrik (disk usage dan memory usage dalam persen) melalui *pipe* ke stdin menggunakan fungsi **scanf**. Jika input yang terbaca bukan 2 angka yang valid, program akan menampilkan pesan *error* kemudian *terminate*.

2. Menentukan status tiap metrik -> setiap nilai diproses oleh fungsi `classify()` yang menentukan status berdasarkan ambang batas varian A, yaitu di mana nilai $\geq 90\%$ menghasilkan status "FAIL", nilai $\geq 75\%$ menghasilkan status "WARN", dan selain kedua nilai tersebut, statusnya adalah "PASS". Fungsi yang sama digunakan untuk kedua metrik (disk usage dan memory usage) karena pada Varian A ambang Disk dan Memory identik.
3. Mengembalikan hasil ke sysinfo.sh -> hasil dikembalikan melalui 2 cara:
 - a. Melalui output teks (DISK <nilai> <status> dan MEM <nilai> <status>) yang ditangkap kembali oleh sysinfo.sh untuk dimasukkan ke tabel laporan
 - b. Melalui exit code sebagai penanda status keseluruhan, di mana 0 jika semua nilai metrik "PASS", 1 jika ada "WARN", 2 jika ada "FAIL", dan 3 jika input tidak valid.

Alasan Penentuan Ambang Metrik 2:

Ambang WARN ditetapkan pada $\geq 75\%$ karena pada titik ini penggunaan memori sudah tinggi dan sistem mulai berisiko melakukan swapping (memindahkan data dari RAM ke disk), yang menurunkan performa. Status WARN berfungsi sebagai peringatan dini agar kondisi dapat diantisipasi sebelum menjadi kritis. Sedangkan, ambang FAIL ditetapkan pada $\geq 90\%$ karena sisa memori sudah sangat sedikit dan sistem berisiko memicu OOM killer, yaitu mekanisme sistem operasi yang mematikan proses secara paksa saat memori habis, sehingga dapat membuat sistem tidak responsif atau *crash*. Penetapan ambang ini juga sejalan dengan konvensi umum pada tool monitoring sistem seperti Nagios dan Icinga, di mana threshold peringatan (*warning*) umumnya berada di kisaran 75–80% dan status kritis (*critical*) pada 90%. Hal ini menunjukkan bahwa ambang yang dipakai mengikuti praktik yang lazim digunakan untuk pemantauan *resource* sistem.

sysinfo.sh**Penjelasan program:**

Skrip ini dijalankan di dalam VM (guest) untuk mengecek kondisi sistem, lalu menggabungkan seluruh hasil pengecekan, termasuk status dari `resource_check.c`, menjadi satu tabel laporan yang disimpan ke `sysinfo_report.txt`.

Alur kerja skrip secara lengkap:

1. Mengecek info OS dan kernel → nama OS diambil dari baris `PRETTY_NAME` pada file `/etc/os-release` menggunakan `grep`, `cut`, dan `tr` untuk membersihkan tanda kutip. Versi kernel diambil dari `uname -r`, lalu dipotong dengan `cut` sehingga hanya menampilkan nomor versi utamanya (contoh: 7.0.0).
2. Menghitung jumlah akun pengguna biasa → file `/etc/passwd` dibaca menggunakan `awk`, dengan memeriksa kolom UID (kolom ketiga). Akun dengan UID di rentang 1000 sampai di bawah 65534 dihitung sebagai akun pengguna biasa, karena UID di bawah 1000 dicadangkan untuk akun sistem dan UID 65534 adalah akun khusus `nobody`.
3. Menghitung jumlah proses berjalan → menggunakan `ps -e --no-headers` untuk menampilkan seluruh proses tanpa baris judul, kemudian jumlah barisnya dihitung dengan `wc -l`.
4. Mendeteksi virtualisasi → menggunakan `systemd-detect-virt`. Karena hasil mentah untuk VirtualBox adalah `oracle`, hasil tersebut diterjemahkan melalui percabangan `case` menjadi VirtualBox agar lebih mudah dibaca. Jika hasilnya `none`, sistem dianggap berjalan di mesin fisik dan

statusnya FAIL; jika terdeteksi, statusnya PASS.

5. Mengambil metrik disk dan memori: Fungsi `get_metrics()` mengambil metrik disk dan memori. Penggunaan direktori / diambil dengan `df -P` / lalu nilainya dibersihkan oleh `awk`. Penggunaan memori dihitung dari `free -b` menggunakan rumus $100 \times (\text{total} - \text{available}) / \text{total}$. Kedua hasil dipastikan berbentuk angka.
6. Mengirim metrik ke program C via pipeline: Fungsi `check_resources()` memastikan `resource_check` tersedia. Nilai disk dan memori dikirim via pipe ke standard input program C, lalu dibaca oleh `scanf`. Program C menentukan status (PASS <75%, WARN 75-89%, FAIL >=90%). Output disimpan ke `RESOURCE_RESULT` dan exit code ke `RESOURCE_CODE` (0=PASS, 1=WARN, 2=FAIL). Status lalu dibaca ke variabel `DISK_STATUS` dan `MEM_STATUS` menggunakan here-string (`<<<`) agar tetap berada di shell yang sama.
7. Fitur tambahan (uptime VM): Fungsi `get_uptime()` membaca durasi boot dari `/proc/uptime`. Nilai tersebut dikonversi `awk` menjadi hari, jam, dan menit menggunakan modulo dan `int()`. Hasilnya disimpan ke variabel `UPTIME`.
8. Menyusun tabel laporan: Fungsi `make_report()` menggabungkan seluruh informasi sistem (OS, user, proses, virtualisasi, disk, memori, uptime). Fungsi `row()` menggunakan `printf` agar lebar kolom sejajar. Seluruh tabel disimpan ke `sysinfo_report.txt` menggunakan pengalihan keluaran `>`.